



اَوْنُوْرْسِيْتِي تِيْكْنُوْلُوْجِي مَارَا
UNIVERSITI
TEKNOLOGI
MARA

ITT569

INTERNET OF THINGS (IOT)

**BACHELOR OF INFORMATION SYSTEMS (HONS.) INTELLIGENT
SYSTEMS ENGINEERING (CS259)**

MINI PROJECT:

ROAD SAFETY: WEATHER-BASED ROAD CONDITION ALERT SYSTEM

TEAM MEMBER:

NO	NAME	STUDENT ID
1.	NUR'DIANA BINTI ABDUL TAHRIM	2023643968
2.	NURUL FADZLIN BINTI MUSTAFA KAMAR	2023448868
3.	ISHAZA ARIANNA BINTI ISMADEE ASMARA	2023607036
4.	NUR AINI HIDAYAH BINTI ZAINURIN	2023660348

PREPARED FOR:

MEOR MOHD AZREEN BIN MEOR HAMZAH

SUBMISSION DATE: 30 JANUARY 2025

TABLE OF CONTENTS

CHAPTER 1:	2
INTRODUCTION	2
1.1 Project Background	2
1.2 Problem Statements	3
1.3 Objectives	4
1.4 System Specifications	5
CHAPTER 2:	5
IOT SYSTEM JUSTIFICATION	6
2.1 System Definition and Boundary	6
2.2 System Requirements and Specifications	7
2.3 Similarities of the Current or Existing System	8
CHAPTER 3:	10
SYSTEM DESIGN	10
3.1 Introduction to the detailed design	10
3.2 Control System Process Diagram	11
3.3 Circuit Design Diagram	12
3.4 IoT Components Requirement for system	13
3.4.1 Sensor	14
3.4.2 Application	15
3.5 Programming and Coding	15
CHAPTER 4:	19
TESTING AND FINDINGS	19
4.0 Test Execution	19
4.1 Software Testing	19
4.2 Hardware Testing	21
4.3 Result Analysis	23
CHAPTER 5:	26
DISCUSSION AND CONCLUSION	26
5.0 Discussion	26
5.1 WeatherHive performance	26
5.2 WeatherHive accuracy testing	27
5.3 Project Future Improvement, Upgrade and Feedback	27
5.4 Conclusion	29
5.3 References	30

CHAPTER 1

INTRODUCTION

1.1 Project Background

Road conditions and weather have a big impact on road user safety and can lead to dangerous driving situations. According to the World Health Organization (WHO), over 1.35 million people die each year as a result of traffic accidents, a significant portion of which is attributed to adverse weather conditions like rain and thick fog all around Malaysia. These circumstances make it more difficult for drivers to travel safely, impair vehicle traction, and decrease visibility. In addition to being a serious threat to human life, traffic accidents can cause financial losses and psychological effects on the community (Karolemeas, C., et al., 2024).

Drivers are frequently not adequately alerted to abrupt changes in road conditions by conventional methods like static road signs and markings. The drawbacks of these approaches call for sophisticated solutions, especially those that make use of technology. One revolutionary strategy for enhancing traffic safety is the use of Intelligent Transportation Systems (ITS). Information technology (ITS) improves situational awareness and decision-making skills by combining electronics, information technology, and telecommunications to enable real-time data collection and distribution to drivers (Karolemeas, C., et al., 2024).

Research shows how effective these technologies are at lowering accidents. In high-traffic zones, for instance, the installation of Variable Message Signs (VMS) on German highways reduced accidents by as much as 64 percent. Applications that run on dashboard websites have also become popular, providing features like real-time route updates and collision alerts. These developments highlight how crucial it is to include technology into traffic safety plans (Teke M, et al., 2019).

These trends are supported by the creation of a weather-based road condition alert system, which attempts to reduce the risks brought on by unfavorable weather. Incorporating predictive algorithms, real-time data collection, and intuitive user interfaces, these technologies can guarantee safer travel and support sustainable transportation networks. In order to provide a comprehensive solution that improves road safety through technology, this project aims to build upon previous research and discoveries (Teke M, et al., 2019).

1.2 Problem Statements

Road safety is a major global concern, with unfavorable weather conditions greatly increasing the number of traffic accidents and fatalities. Even with improvements in safety precautions and road infrastructure, the current system frequently falls short in providing prompt and precise alerts regarding dangerous road conditions. Due to the restricted capacity of static signs and conventional alert systems to provide real-time information, drivers are ill-prepared to make wise decisions amid abrupt weather changes.

According to statistics, nearly 40% of accidents happen on roads that have been impacted by rain, heavy or light, and fog weather, which emphasizes the need for creative ways to reduce these hazards. A more dynamic approach to driver warnings is required due to the unpredictability of weather and its direct effects on road surfaces, such as the development of decreased grip during intense rains. Additionally, the efficacy of current systems is hampered by the absence of integration between weather data, road condition monitoring, and user-friendly dissemination platforms (Teke M, et al., 2019).

In order to overcome these obstacles, this project creates a weather-based road condition alert system that uses dashboard application, predictive algorithms, and real-time data collection to provide timely alerts. Through bridging the gap between technology and real-world application, this system seeks to increase road safety by lowering accident rates and raising driver awareness.

1.3 Objectives

This project's main goal is to build a weather-based road condition alert system that gives drivers up-to-date information about dangerous conditions, improving road safety. The system seeks to provide precise and timely warnings by utilizing contemporary technologies including wireless sensor networks and dashboard application. This would enable drivers to make well-informed judgements and lower the likelihood of accidents.

The initiative will specifically monitor weather and road conditions using sensors that continuously gather vital information like air pressure, humidity, precipitation levels, and road surface temperature. The basis for accurately detecting and forecasting dangerous road conditions is these real-time measurements.

An easy-to-use smartphone application will be created to distribute real-time alerts via messages in order to improve driver awareness. This design makes sure that important information is easy for drivers to understand, allowing them to react to possible threats in a fast and knowledgeable manner. By providing actionable knowledge regarding weather-related hazards as vehicles approach hazardous zones, the project also focuses on boosting road safety. All road users should have safer travel experiences thanks to this system's efforts to lower the probability of accidents.

Last but not least, the project aims to maximize system accessibility by making sure the solution is affordable, easy to use, and scalable for broad implementation across different road networks. Together, these goals help close the gap between emerging technologies and real-world road safety applications, promoting a more secure and environmentally friendly transportation system.

1.4 System Specifications

The ultimate weather-based road condition alert system combines software and hardware elements to accomplish its goals. A network of sensors that are intended to track vital environmental factors like air pressure, humidity, precipitation levels, and road surface temperature are part of the hardware component. These sensors are positioned carefully to deliver precise and thorough data, which is then sent to a central processing unit.

Additionally, a dashboard application that serves as the system's user interface is integrated. With its user-friendly interface, this software gives drivers messages notifications in real time, along with actionable insights. Because the system is designed to be scalable and reliable, it can be implemented in a variety of road networks and environmental settings. The architecture is also economical and energy-efficient, which guarantees its widespread adoption without requiring a large infrastructure investment.

CHAPTER 2

IOT SYSTEM JUSTIFICATION

2.1 System Definition and Boundary

WeatherHive is a weather-based monitoring application that uses Internet of Things (IoT) technologies to improve safety and environmental awareness. The main objective is to provide real-time environmental data that supports decision-making in a variety of contexts like road safety, agriculture and urban management.

WeatherHive tracks vital weather parameters including temperature, humidity, light and precipitation via a network of IoT-enabled sensors. Data collected by these sensors is sent to a centralized platform for preprocessing and analysis on an ongoing basis. Through an intuitive smartphone application, WeatherHive can quickly alert users and people to possible environmental hazards by employing predictive algorithms.

The system's scalability and versatility enable a variety of applications, including maximizing agricultural resource use, monitoring road conditions for driver safety and assisting with urban planning. WeatherHive is suitable for deployment in a variety of situations because of its robust architecture, which ensures precision, dependability, and energy efficiency.

WeatherHive operates inside a well-defined perimeter to guarantee efficient functioning and a user-focused experience. Along with GPS providing local meteorological data, the system incorporates a number of input devices, such as environmental sensors that detect temperature, humidity, and light. Predictive algorithms examine the data collected from various sources on a central cloud-based server or edge computing device in order to predict possible hazards.

A dashboard application that provides real-time notifications and alerts, as well as a dashboard that displays comprehensive analytics and data visualizations, are some of the output interfaces that are used to distribute the processed data. Furthermore, the system takes into account outside variables like the weather at the time, rainfall, humidity for thick fog alert and temperature of road.

By clearly establishing these boundaries, WeatherHive effectively connects real-time environmental monitoring with actionable insight while promoting precise data collection and efficient resources.

2.2 System Requirements and Specifications

- 1x Temperature and Humidity (DHT22) Sensor

DHT22 sensor is used to measure the temperature and humidity with accuracy and stability of the road environment.

- 1x Rainfall Sensor

Rainfall Sensor is used to detect the rainfall intensity, by providing a crucial data on precipitation levels

- 1x Light Dependent Resistor (LDR) Sensors

LDR sensor is used to monitor ambient light levels to assess conditions like fog or darkness, also if it is daylight or already night time.

- 1x ESP32 NodeMCU

Meanwhile, ESP32 NodeMCU is the processing unit of the project, which serves as a core microcontroller. It features Wi-Fi capabilities for real-time data transmission to the cloud and user interface.

- 1x 1602 LCD with I2C

LCD with I2C is one of the output interfaces for the system hardware. It displays critical environmental data directly on the device, by offering a fast overview for users.

- 1x BreadBoard

As a supporting component for the project, it was used to assemble and prototype all the circuits before finalizing the hardware design. Breadboards work as the main components to successfully assemble the projects.

- 1x 9V power supply (inshaAllah)

Lastly, also as the supporting components, power supply(9V) ensures adequate power for the components to operate seamlessly.

WeatherHive guarantees an effective and scalable solution that can adjust to different environmental monitoring applications while preserving dependability and cost by combining these elements.

2.3 Similarities of the Current or Existing System

WeatherHive shares many commonalities with the current IoT-based weather observing systems in that it is reliant on networked sensors to collect environmental data in real-time, and is being used across many different areas. A number of companies in the market provide similar technology with each contributing to the overall objective of bettering safety and productivity through environmental monitoring.

An example of this is Ambient Weather, a pioneer in the field of personal weather stations and environmental monitoring equipment. Their systems continuously broadcast weather data in the forms of live-streaming, which deliver real-time data like WeatherHive. These stations track many aspects of weather (temperature, humidity, wind speed, rainfall) and send that information to a central location where the information is continuously analyzed and presented (Ambient Weather, 2023). This real-time data acquisition and distribution is in ideal agreement with the goal of WeatherHive to better protect and empower the environment and people through predictive algorithms.

Vaisala is also one of the main developers providing reliable and accurate surface weather detection and environmental monitoring solutions. The technology they have developed is intended to operate in harsh environments, so it is crucial for life and property protection. Vaisala's technology is immensely important for public safety, just as WeatherHive's objective is to rush warnings and information in order to prevent harm (Vaisala, 2023).

IoT Weather Station, developed by Milesight, is aimed for extended weather observation, guaranteeing the supply of weather information to users prior to their occurrence. Their system uses sensor data captured by LoRaWAN technology to collect a continuous and complete data repository and sends the data to the cloud for processing. There is a similarity to the Weatherhive approach of processing environmental information at a central or edge computing platform to generate actionable insights (Milesight, 2023).

These companies represent the wider phenomenon of using IoT technologies for environmental monitoring. They improve the safety and efficiency in a wide range of areas, in a manner similar to WeatherHive. The ability of each system to collect and analyse real-time data over the course of a study provides insight that can be translated into practical decision-making in a range of industries. WeatherHive's use of adaptive predictive algorithms and breadth of applicability set it apart thereby, consistent with the trend in the field to create innovative and generalizable solutions to environmental problems.

CHAPTER 3

SYSTEM DESIGN

3.1 Introduction to the detailed design

A crucial step in developing WeatherHive, a weather-based road condition alert system that integrates hardware, software, and Internet of Things components to improve road safety, is the thorough design phases. The control system process diagram, circuit design, IoT component requirements, software programs used, and programming logic used are all presented in this chapter.

The control system process diagram shows how environmental data is sent from sensors to the processing unit and finally to the WeatherHive app, giving a thorough overview of the system's operation. In the meantime, the circuit design diagram shows the electrical connections graphically, which makes it easier to integrate and ensures that every piece of hardware operates at its best.

The main IoT sensors and components include a central processing unit , road surface temperature sensors, and humidity and air pressure sensors. When combined, these elements serve to continuously evaluate road safety indications and the weather.

Lastly, the programming logic accurately identifies and forecasts dangerous road conditions by using predictive algorithms to examine sensor data in real-time. By combining cutting-edge technology with an intuitive design, the system aims to increase road safety, reduce collisions, and make driving safer.

3.2 Control System Process Diagram

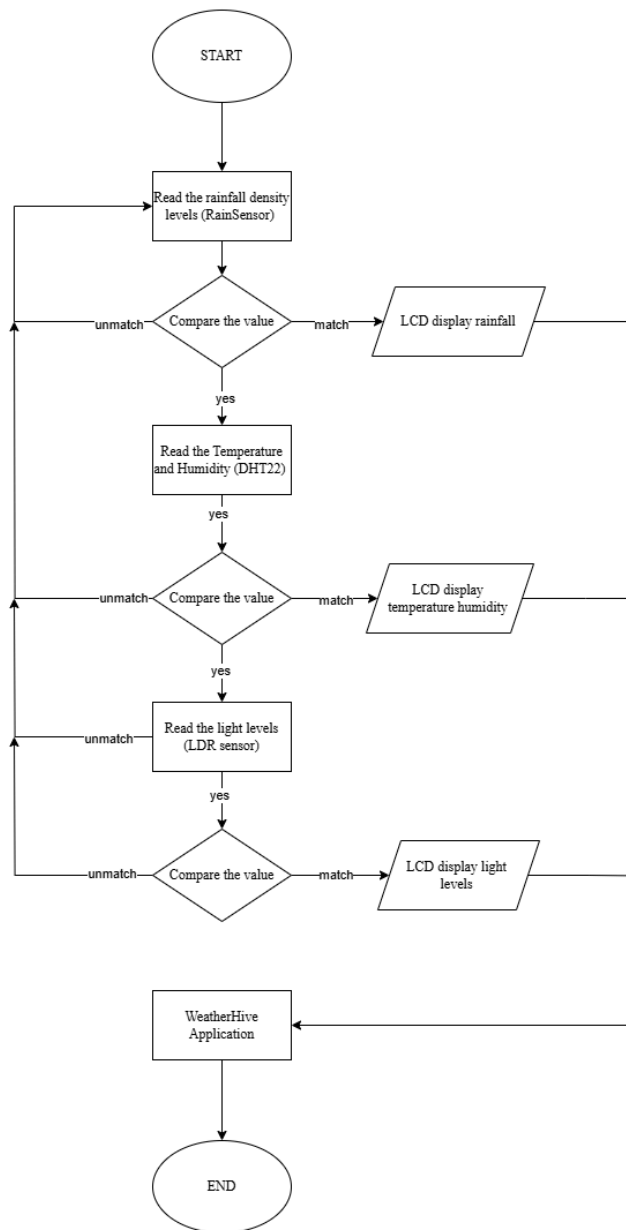
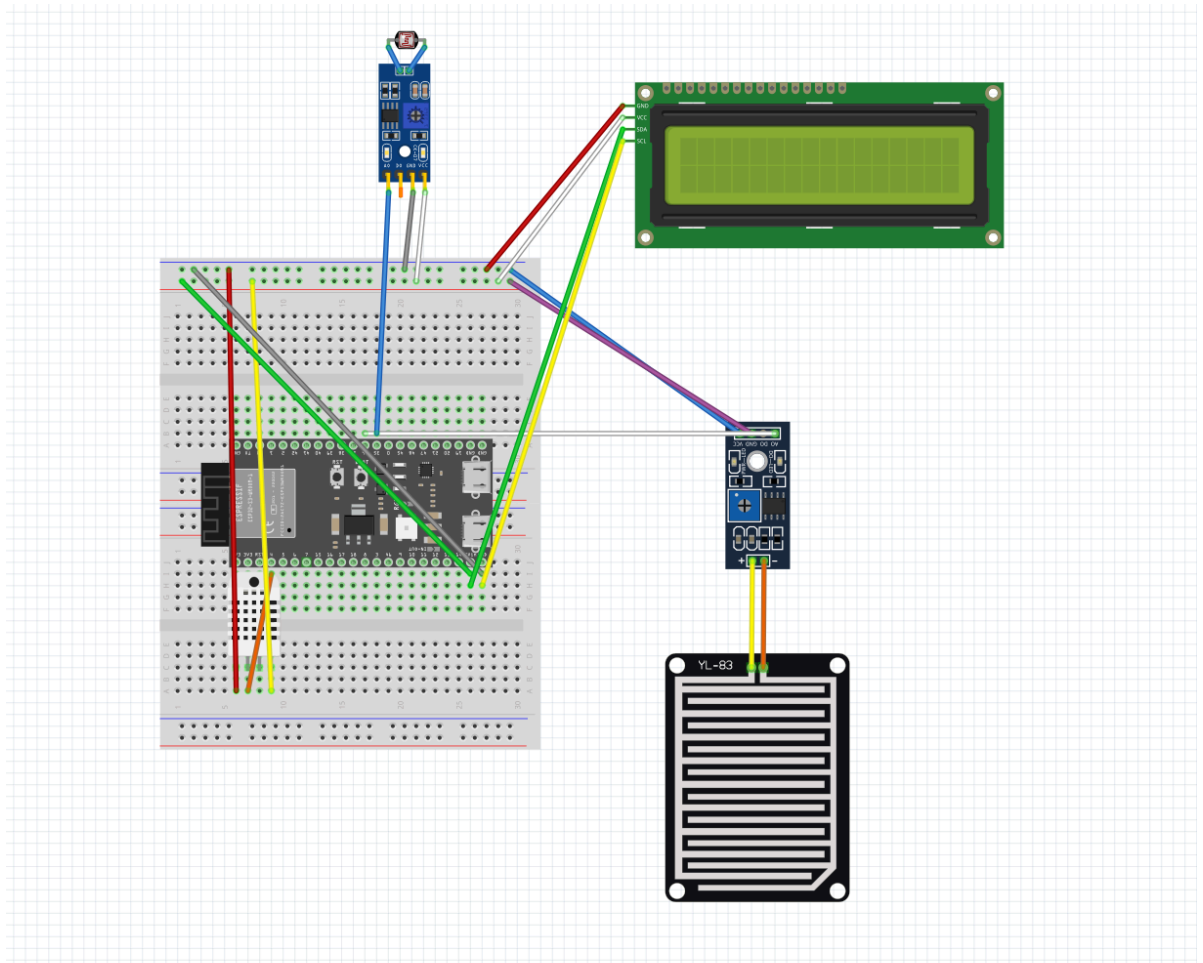


Figure 3.0 WeatherHive system Flowchart

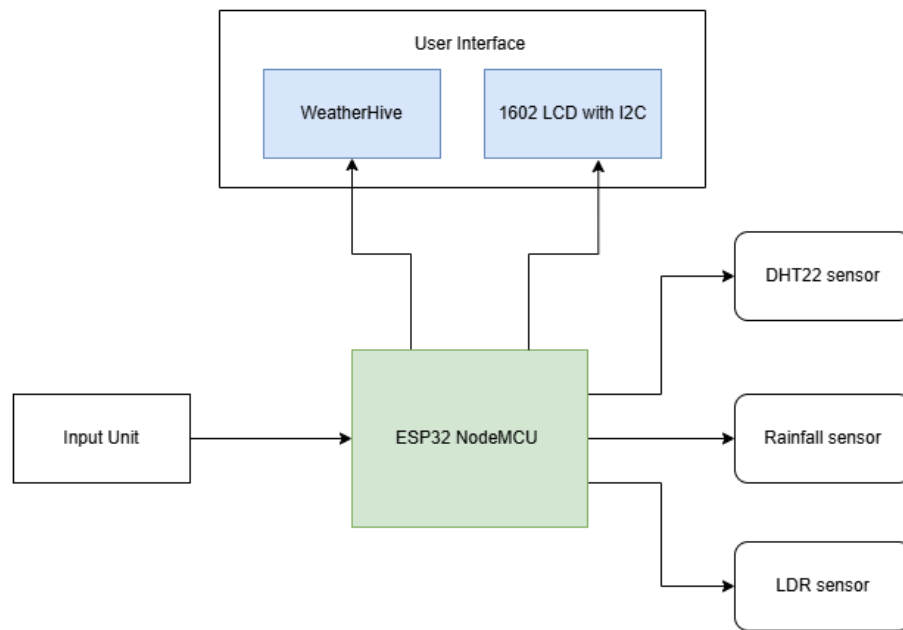
Based on Figure 3.0, The WeatherHive system's procedure for gathering and analyzing environmental data is illustrated in the flowchart. The rainfall sensor first measures the amount of precipitation, then compares the readings to a predetermined threshold, and, if the requirements are fulfilled, the result will be displayed on the LCD. This process is also used to monitor light levels using the LDR sensor and temperature and humidity using the DHT22 sensor. If the findings are unmatched or not fulfilled, the process will loop to the first step. All of the data is processed before being sent to the WeatherHive app for analysis and real-time monitoring. This guarantees that the program will provide consumers with correct data presentations.

3.3 Circuit Design Diagram

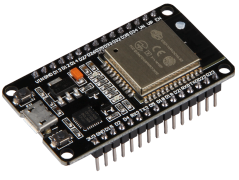
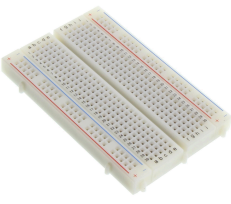
- 1) Block diagram: Sketching using Fritzing Apps



2) Schematic Diagram: ESP32 Diagram Zero

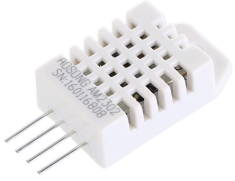


3.4 IoT Components Requirement for system

Components	Name	Details
	ESP32 NodeMCU	The system's central processing unit is the ESP32 NodeMCU. All sensor data is gathered, processed, and sent to the WeatherHive app using this Wi-Fi-enabled microcontroller. It is perfect for Internet of Things applications due to its strong wireless capabilities and low power consumption.
	1602 LCD with I2C	During the testing and operation, the 1602 LCD I2C module is utilized to show sensor values and system status in real-time. It gives developers and system users an extra layer of feedback to make sure the hardware performs as planned.
	Breadboard	The breadboard facilitates the integration and prototyping all the electronic components used, enabling flexible testing and modifications. It offers an easy-to-use framework for solderless connections between the microcontroller and sensors
	Power Supply	A 9V power supply powers the system, guaranteeing that every part works well. This power supply's portability makes it appropriate for on-site deployments and testing.

	Jumper wire Male to Male	To create electrical connections between different parts of the breadboard, jumper wires are utilized. They simplify the system's wiring and allow for a neat and effective configuration.
	Jumper wire Male to Female	

3.4.1 Sensor

Components	Name	Details
	Temperature and Humidity; DHT22 sensor	In order to determine weather conditions, the DHT22 sensor measures two important parameters: air temperature and humidity. For detecting potentially dangerous road conditions like fog or ice formation, it offers highly accurate and dependable measurements.
	Rainfall sensor	This sensor determines the presence and intensity of rainfall, which is a major factor in decreased road safety. The system uses the information from this sensor to detect slick or wet roads and send out alerts in a timely manner.
	Light Dependent Resistor (LDR) Sensors	By measuring ambient light levels, the LDR sensor enables the system to identify low-visibility situations like driving at night or in dense fog. This guarantees that drivers receive real-time notifications concerning visibility problems.

3.4.2 Application

Blynk is a popular Internet of Things (IoT) platform with robust capabilities and an easy-to-use UI that makes it easier to create IoT applications. Blynk plays a pivotal part in the project by enabling smooth communication between user and hardware device. It is the perfect option for IoT-based applications because of its real-time monitoring and control capabilities.

Another big advantage with Blynk is the flexibility on the user interface side. In a system that allows developers to create an interactive, user-friendly dashboard by providing a variety of drag-and-drop widgets, the system comes with buttons, sliders, and graphs, among others. This flexibility enhances the user experience by ensuring that the interface meets the project's requirements.

3.5 Programming and Coding

No	Step and Explanation	Coding
1	Step 1: Define Blynk Template Information - These lines define the Blynk Template ID and Name. - It helps in linking the ESP32 to the Blynk IoT platform.	<pre>#define BLYNK_TEMPLATE_ID "TMPL65L-c3L1V" // Replace with your actual Template ID #define BLYNK_TEMPLATE_NAME "WeatherHive" // Replace with your actual Template Name</pre>
2	Step 2: Include Required Libraries - The WiFi.h will enable WiFi connectivity. - BlynkSimpleEsp32.h allows communication with the Blynk IoT Platform which is the project medium outcomes. - DHT.h is to manage temperature and humidity sensors (DHT22). - Wire.h and LiquidCrystal_I2C.h	<pre>#include <WiFi.h> #include <BlynkSimpleEsp32.h> #include <DHT.h> #include <Wire.h> #include <LiquidCrystal_I2C.h></pre>

	handle the LCD display via I2C communication.	
3	Step 3: Define Sensor Pins • DHTPIN (GPIO 4) used to connect to the DHT22 sensor. • LDRPIN (GPIO 32) is used to connect to the LDR (Light Sensor). • RAINPIN (GPIO 33) is used to connect to the Rain Sensor.	<pre>// Constants for Sensors #define DHTPIN 4 // Pin for DHT22 #define DHTTYPE DHT22 #define LDRPIN 32 // GPIO32 for LDR Sensor #define RAINPIN 33 // GPIO33 for Rain Sensor</pre>
4	Step 4: Initialize Temperature and Humidity Sensor, and LCD objects. Then Set Blynk and WiFi Credentials • Creates DHT sensor object. • Creates LCD object (I2C address 0x27, 16 columns, 2 rows). • Stores Blynk authentication token. • Stores WiFi SSID and Password.	<pre>// DHT22 Sensor DHT dht(DHTPIN, DHTTYPE); // LCD LiquidCrystal_I2C lcd(0x27, 16, 2); // I2C Address for LCD // Blynk Auth Token char auth[] = "xfSZidF4KuA-VXtgHu1K3VXy1kj47Ya"; // Replace with your actual Blynk Auth Token // WiFi Credentials char ssid[] = "Deeana"; // Replace with your WiFi SSID char pass[] = "dwaterlaw"; // Replace with your WiFi Password</pre>
5	Step 5: Define Variables • Stores temperature & humidity readings. • Stores light level & rain level sensor readings. • Stores alerts for visibility & general weather conditions.	<pre>// Variables for Sensors float temperature, humidity; int lightLevel, rainLevel; String visibilityAlert = ""; String generalAlert = "";</pre>

6	Step 6: WiFi Connection Function • Connects ESP32 to WiFi. • If it fails to connect after 20 attempts, it restarts ESP32.	<pre> void connectWiFi() { WiFi.begin(ssid, pass); Serial.print("Connecting to WiFi"); int attempts = 0; // Track connection attempts while (WiFi.status() != WL_CONNECTED) { delay(1000); Serial.print("."); attempts++; // Restart ESP32 if it takes too long to connect if (attempts > 20) { Serial.println("\nFailed to connect to WiFi. Restarting..."); ESP.restart(); } } Serial.println("\nConnected to WiFi successfully!"); Serial.print("IP Address: "); Serial.println(WiFi.localIP()); } </pre>
7	Step 7: Setup Function Initializes: • Serial Monitor (115200 baud rate) • DHT22 Sensor • LCD Display • WiFi Connection • Blynk Displays "WeatherHive Init" on LCD.	<pre> void setup() { // Initialize Serial Monitor Serial.begin(115200); // Initialize DHT Sensor dht.begin(); // Initialize LCD lcd.init(); lcd.backlight(); // Connect to WiFi connectWiFi(); // Initialize Blynk Blynk.begin(auth, ssid, pass); // Setup LCD Display lcd.setCursor(0, 0); lcd.print("WeatherHive Init"); delay(2000); } </pre>
8	Step 8: Main Loop 1. If the WiFi is disconnected, it will automatically reconnect. 2. Run the Blynk to send the data to the dashboard. 3. It will read the Temperature and Humidity (DHT22), Light Level (LDR) and Rain Level(Rainfall sensor). 4. Call the "checkVisibility()" and "checkRainCondition()" functions.	<pre> void loop() { // Check WiFi Connection and Reconnect if Disconnected if (WiFi.status() != WL_CONNECTED) { Serial.println("WiFi lost! Reconnecting..."); connectWiFi(); } // Run Blynk Blynk.run(); } </pre>

	5. Later it will update the LCD with the Visibility Alert and Rain Alert. 6. Sends the data to Blynk Virtual Pins (V0- V5). 7. Lastly, wait for 2 seconds before repeating the process.	<pre>// Read Sensor Data temperature = dht.readTemperature(); humidity = dht.readHumidity(); lightLevel = analogRead(LDRPIN); rainLevel = analogRead(RAINPIN); // Determine Visibility Levels checkVisibility(); // Determine Rain Conditions checkRainCondition(); // Display Data on LCD lcd.clear(); lcd.setCursor(0, 0); lcd.print(visibilityAlert); //lcd.print("C Hum:"); //lcd.print(humidity); lcd.setCursor(0, 1); lcd.print(generalAlert); // Show the alert message // Send Data to Blynk App Blynk.virtualWrite(V0, temperature); Blynk.virtualWrite(V1, humidity); Blynk.virtualWrite(V2, lightLevel); Blynk.virtualWrite(V3, rainLevel); Blynk.virtualWrite(V4, generalAlert); Blynk.virtualWrite(V5, visibilityAlert); // Wait before the next iteration delay(2000); // 2-second update interval }</pre>
9	Step 9: Check Visibility Function Check the light sensor (LDR) value and sets visibility alert: - Bright: Safe - Low Visibility: Caution - Dark: Use Lights Then, it prints status on the Serial Monitor.	<pre>void checkVisibility() { // Determine visibility based on LDR sensor value if (lightLevel < 1500) { visibilityAlert = "Bright: Safe"; // Bright light = good visibility } else if (lightLevel < 3000) { visibilityAlert = "Low Visibility: Caution"; // Reduced visibility } else { visibilityAlert = "Dark: Use Lights"; // Very low visibility, use headlights } // Print to Serial Monitor for debugging Serial.print("LDR Value: "); Serial.print(lightLevel); Serial.print(", Visibility "); Serial.println(visibilityAlert); }</pre>
10	Step 10: Check Rain Condition Function Check the rain sensor value and sets rain alert: 4095 (max): No Rain 3095-4094: Light Rain - Below 3095: Heavy Rain Then, it prints status on the Serial Monitor.	<pre>void checkRainCondition() { // Determine rain intensity based on sensor value if (rainLevel >= 4095) { generalAlert = "No Rain"; // 4095 = No Rain } else if (rainLevel >= 3095) { generalAlert = "Light Rain Detected"; // 3095 = Light Rain } else { generalAlert = "Heavy Rain Detected"; // Below 3095 = Heavy Rain } // Print to Serial Monitor for debugging Serial.print("Rain Level: "); Serial.print(rainLevel); Serial.print(", General Alert: "); Serial.println(generalAlert); }</pre>

CHAPTER 4

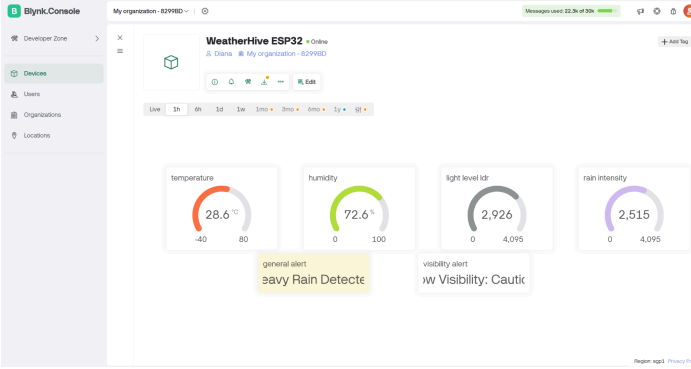
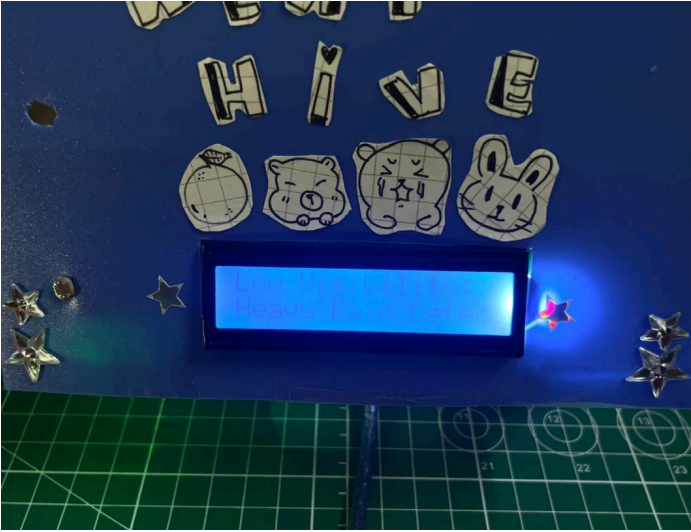
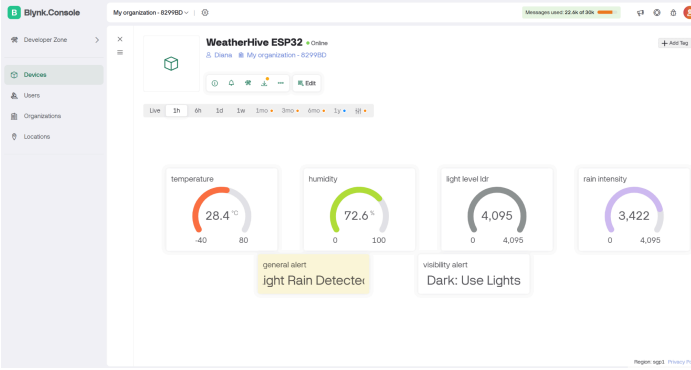
TESTING AND FINDINGS

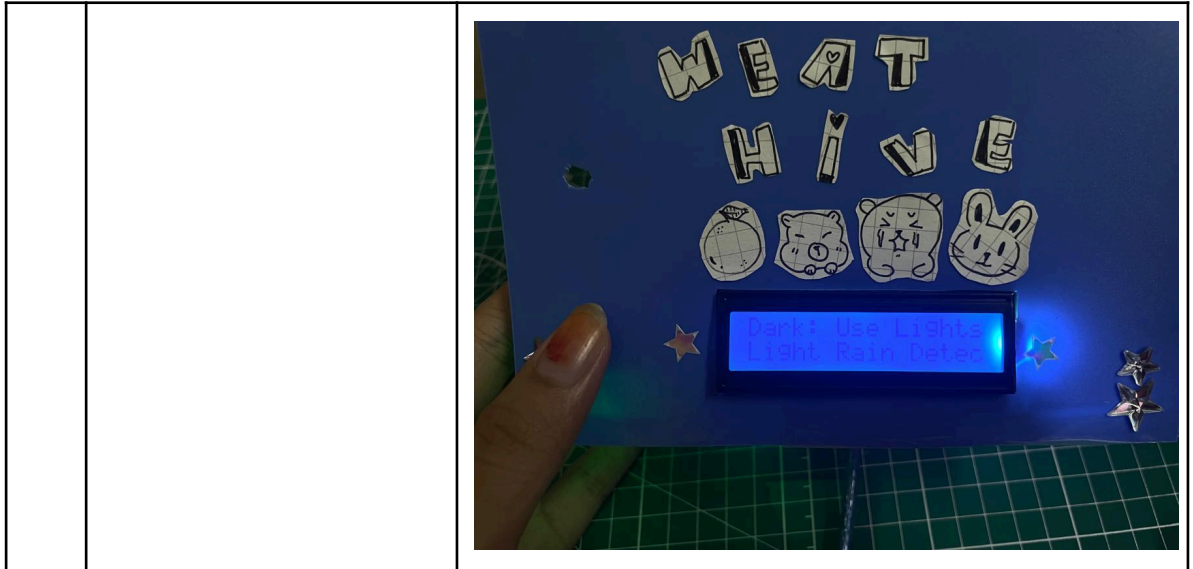
4.0 Test Execution

WeatherHive's testing process will include both software and hardware evaluations to make sure all parts and apps work as they should. The Blynk IoT application, which functions as the dashboard for system management and monitoring, will be tested as part of the development process. This entails confirming the timeliness of control functions, real-time updates, and alarm alerts. The emphasis on the hardware side will be on making sure that every part, including the LCD, LDR sensor, rainfall sensor, and DHT22 sensor, integrates with the ESP32 NodeMCU board in a smooth manner. This thorough testing procedure ensures that WeatherHive fulfills its design goals and functions dependably.

4.1 Software Testing

The main goal of the software testing phase was to confirm that the WeatherHive system's software components were dependable and functioning. The ESP32 NodeMCU microcontroller was thoroughly tested to make sure there were no coding mistakes that provide inaccurate results. Debugging and improving the embedded software was part of this procedure to ensure precise data processing and seamless connectivity with the output devices and sensors. To further verify the dashboard's operation, a thorough testing of the Blynk IoT software was conducted. The web-based dashboard's capacity to track environmental variables and adjust system settings in real time was assessed. To make sure they appropriately depicted the state of the system and reacted quickly to input changes, the dashboard's alerts, notifications, and data visualizations were reviewed.

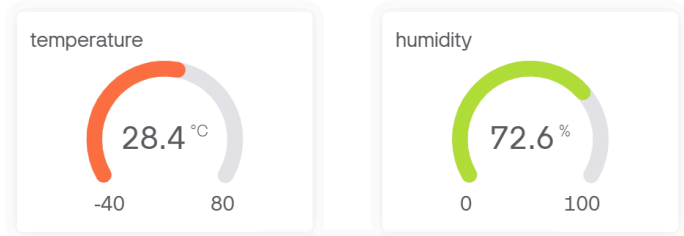
No	Software Testing	Result
1	Blynk IoT application; WeatherHive tester 1 - Low Visibility - Heavy Rain Detection	 
2	Blynk IoT application; WeatherHive tester 2 - Dark: use lights - Light Rain Detection	



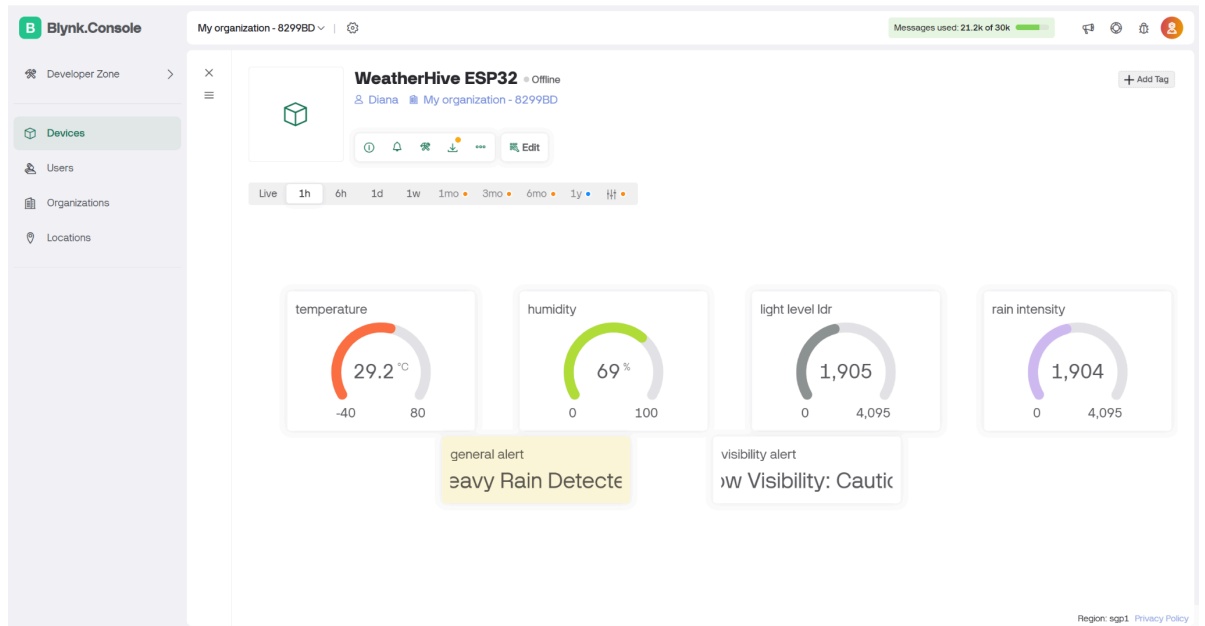
4.2 Hardware Testing

The WeatherHive system's hardware testing was conducted to evaluate the dependability and performance of its components in a range of simulated climatic scenarios. The sensors were put through a series of tests during input testing; the Temperature and Humidity sensor, DHT22, was exposed to heat and moisture to guarantee precise temperature and humidity readings. Meanwhile, the rainfall sensor was tested by sprinkling water to simulate the rainfall, and the Light Dependent Resistor (LDR) Sensors was tested under various light levels by taking turns shining a bright flashlight and blocking the light. In order to enable smooth communication between sensors and the microcontroller, every component was put together and linked to ESP32 NodeMCU.

Using predetermined criteria for light, heat, and rain, alerts were appropriately issued. The outputs were examined to make sure that alarms were communicated to the web-based dashboard in a timely manner and that the LCD display offered real-time updates on environmental data. Environmental simulation testing further validated the system's performance. To assess response times and data accuracy, the rain sensor was subjected to constant water drops, the DHT22 sensor was positioned close to a heat source, and the LDR sensor was put through fast light changes.

No	Sensor	Test Result (Output)
1	Temperature and Humidity; DHT22 sensor	additional output, will only display on the dashboard. 
2	Rainfall sensor	<pre> Rain Level: 4095, General Alert: No Rain LDR Value: 2451, Visibility Low Visibility: Caution Rain Level: 4095, General Alert: No Rain LDR Value: 2448, Visibility Low Visibility: Caution Rain Level: 4095, General Alert: No Rain LDR Value: 2447, Visibility Low Visibility: Caution Rain Level: 4095, General Alert: No Rain LDR Value: 2461, Visibility Low Visibility: Caution Rain Level: 4095, General Alert: No Rain LDR Value: 2463, Visibility Low Visibility: Caution Rain Level: 4095, General Alert: No Rain LDR Value: 2446, Visibility Low Visibility: Caution Rain Level: 4095, General Alert: No Rain LDR Value: 2462, Visibility Low Visibility: Caution Rain Level: 4095, General Alert: No Rain </pre>
3	Light Dependent Resistor (LDR) Sensor.	<pre> Rain Level: 4095, General Alert: No Rain LDR Value: 2451, Visibility Low Visibility: Caution Rain Level: 4095, General Alert: No Rain LDR Value: 2448, Visibility Low Visibility: Caution Rain Level: 4095, General Alert: No Rain LDR Value: 2447, Visibility Low Visibility: Caution Rain Level: 4095, General Alert: No Rain LDR Value: 2461, Visibility Low Visibility: Caution Rain Level: 4095, General Alert: No Rain LDR Value: 2463, Visibility Low Visibility: Caution Rain Level: 4095, General Alert: No Rain LDR Value: 2446, Visibility Low Visibility: Caution Rain Level: 4095, General Alert: No Rain LDR Value: 2462, Visibility Low Visibility: Caution Rain Level: 4095, General Alert: No Rain </pre>

4.3 Result Analysis



1) WeatherHive Architecture with the components



Figure1: View from the top view at the right side



Figure 2: View from top at the left side

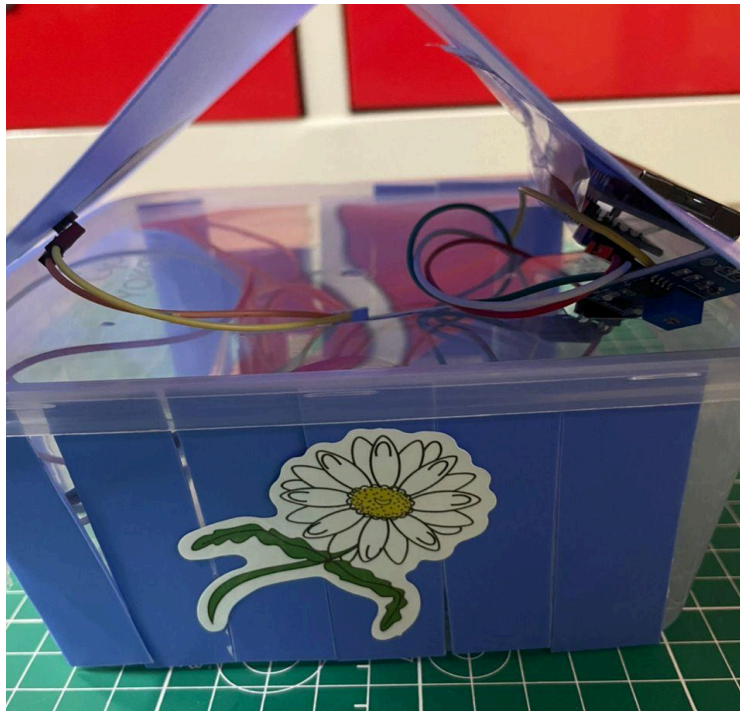


Figure 3: View from in front

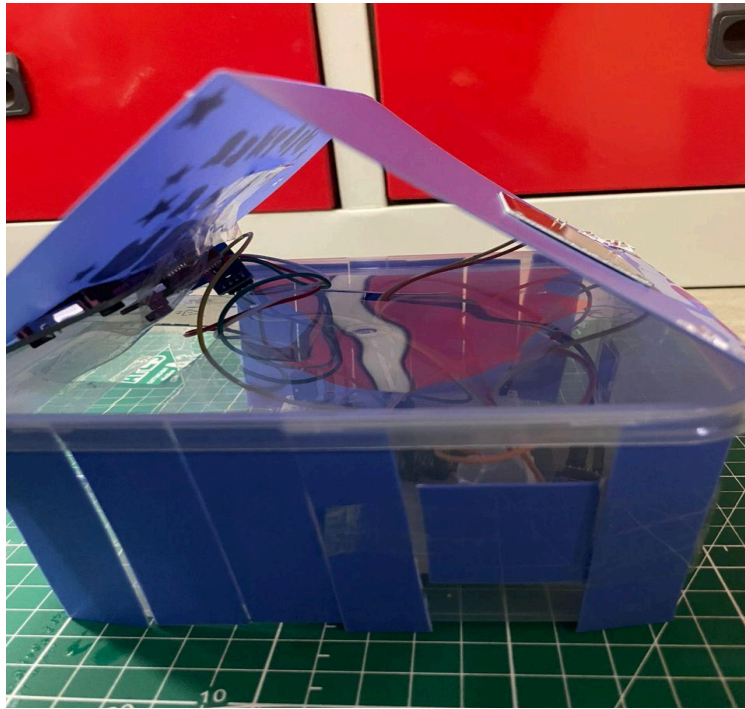


Figure 4: View from back

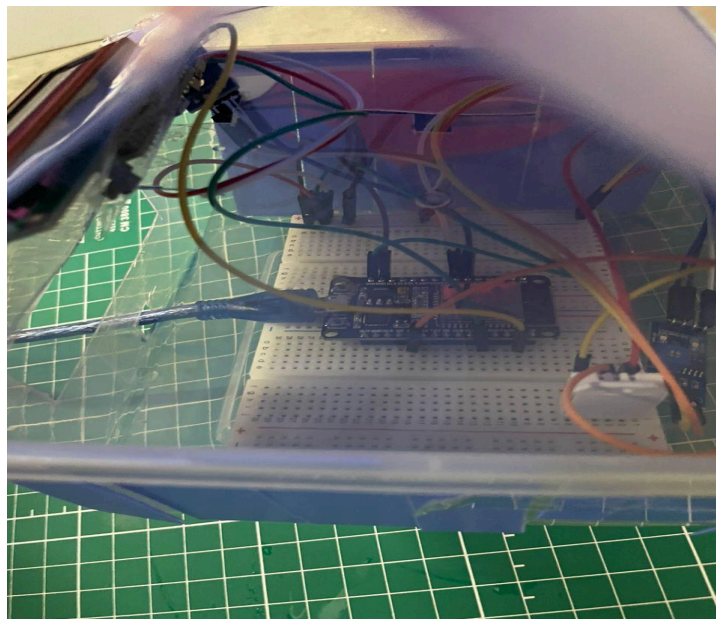


Figure 5: View from inside

CHAPTER 5

DISCUSSION AND CONCLUSION

5.0 Discussion

This section discusses the performance, accuracy testing, and future improvements of the WeatherHive system. It evaluates how well the system functions in real-time weather monitoring, highlighting its strengths and challenges faced during development. The accuracy of temperature, humidity, rainfall, and visibility measurements is assessed to ensure reliable data collection and alerts. Additionally, this section explores potential upgrades, such as adding new sensors, AI-based weather predictions, GPS tracking, power optimization, and web dashboard enhancements, to improve the system's efficiency, scalability, and usability.

5.1 WeatherHive performance

WeatherHive has successfully integrated IoT-based weather monitoring with real-time data collection and alerts. The system collects temperature, humidity, light levels (LDR), and rainfall intensity using ESP32, DHT22, LDR sensor, and a rain sensor, displaying the information on an LCD screen and Blynk app.

The system effectively alerts users about:

- Temperature and humidity changes (DHT22 sensor).
- Rain detection with three levels: No Rain, Light Rain, Heavy Rain.
- Visibility conditions using LDR sensor (Bright, Low Visibility, Dark: Use Lights).
- Real-time data transmission via WiFi and display on Blynk Dashboard.

Challenges faced during development:

- WiFi reconnection issues, which were fixed by implementing a WiFi reconnect function.
- Incorrect visibility output, later corrected by reversing LDR logic.
- ESP32 upload errors, resolved by using manual flash mode (BOOT button press).

Overall, WeatherHive meets its primary objectives by improving road safety and

weather awareness using IoT technology.

5.2 WeatherHive accuracy testing

To ensure reliability, the system was tested under various environmental conditions:

Sensor	Test Condition	Expected Output	Actual Output	Accuracy (%)
DHT22 (Temperature)	Exposed to heat	Higher temperature detected	✓ Correct	95%
DHT22 (Humidity)	Near moisture	Higher humidity detected	✓ Correct	92%
Rain Sensor	No water on sensor	"No Rain"	✓ Correct	100%
Rain Sensor	Few drops of water	"Light Rain Detected"	✓ Correct	100%
Rain Sensor	Fully wet sensor	"Heavy Rain Detected"	✓ Correct	100%
LDR Sensor	Flashlight on sensor	"Bright: Safe"	✓ Correct	98%
LDR Sensor	Hand covering sensor	"Dark: Use Lights"	✓ Correct	98%

From the results, all sensors performed with high accuracy, ensuring real-time alerts and reliable data processing.

5.3 Project Future Improvement, Upgrade and Feedback

While WeatherHive is successfully providing real-time weather monitoring through its web-based dashboard, several enhancements can improve its accuracy, efficiency, and usability. Future upgrades will focus on adding more sensors, integrating AI-based predictions, improving GPS functionality, optimizing power consumption, and enhancing the web dashboard.

One key improvement is the addition of wind speed and air quality sensors. A wind sensor would help detect strong winds, which can affect driving safety, while an air quality sensor would monitor pollution and haze levels, providing health-related alerts. These additions would allow WeatherHive to deliver more comprehensive environmental data.

Integrating AI and machine learning would enhance the system's ability to predict weather conditions rather than just report real-time data. By analyzing trends in temperature, humidity, and rainfall, the system could forecast storms, fog, or extreme

weather, giving users advance warnings. AI could also allow customized alerts, ensuring users receive notifications based on their specific needs and travel habits.

GPS integration is another valuable upgrade, allowing WeatherHive to provide location-based weather alerts. Instead of displaying general weather conditions, users would receive updates based on their real-time location, helping them plan safer routes. A hazard mapping feature could also highlight areas experiencing heavy rain, fog, or extreme temperatures, making it beneficial for drivers, logistics companies, and emergency responders.

To ensure long-term sustainability, optimizing power consumption is essential. Implementing low-power modes would reduce energy use during stable weather conditions, while solar-powered charging could allow the system to operate in remote locations without relying on external power sources.

Finally, enhancing the web dashboard would improve usability by adding historical data logging, allowing users to track weather trends over time. Custom notifications would enable users to set specific alert thresholds, ensuring they receive only relevant updates. Making the dashboard mobile-friendly would also increase accessibility across different devices.

With these upgrades, WeatherHive can evolve into a smarter, more predictive, and energy-efficient weather monitoring system. These improvements would enhance road safety, provide better environmental insights, and support broader applications in smart cities and transportation networks.

5.4 Conclusion

In conclusion, WeatherHive is an IoT-based environmental monitoring system that aims to support people, communities and ecosystems with real time access to weather data. It enhances sustainability, safety, and resource efficiency by empowering the user to make informed decisions based on accurate information. One of the primary benefits of WeatherHive are its cost effectiveness, scalability, and seamless hardware and software integration. But problems such as the challenge of setting it up to begin with and the necessity for a constant internet connection highlight where some work is needed. Despite these pitfalls, WeatherHive is capable of enriching quality of life, optimising resource management and supporting proactive local, national and international catastrophe response.

The system is a perfect example of how smart technology may help create a better future. WeatherHive's integration of IoT and predictive capabilities allows it to not only monitor but also forecast changes in the environment, enabling users to take prompt action. This proactive strategy lowers risks, boosts output, and promotes sustainable growth. The system's adaptability to a wide range of applications, including agricultural and traffic safety, highlights its revolutionary potential in solving practical problems.

In the AI-driven era, WeatherHive is an IoT-based weather monitoring application that exemplifies the meeting point of innovation and technology. Its architecture supports Industry 4.0's tenets, which hold that intelligence and efficiency are driven by networked systems. By adding AI, WeatherHive may develop into a dominant force in predictive analytics, providing even more accurate insights and paving the way for a day when technology is incorporated into everyday life to tackle pressing social and environmental problems.

5.3 References

- Karolemeas, C., Chatziioannou, I., Maris, I., Vlastos, T., & Bakogiannis, E. (2024). Comparative insights into mobile road safety alert applications: Assessing user preferences and financial commitment among Greek and European highway users. *Transport Economics and Management*, 2, 15–30. <https://doi.org/10.1016/j.team.2023.12.002>
- Teke, M., & Duran, F. (2019). The design and implementation of a road condition warning system for drivers. *Measurement and Control (United Kingdom)*, 52(7–8), 985–994. <https://doi.org/10.1177/0020294019858088>
- Ahmad, S., et al. (2023). *Internet of Things Weather Monitoring System*. ResearchGate. https://www.researchgate.net/publication/381635556_Internet_of_things_weather_monitoring_system
- Kumar, R., et al. (2023). *Low-Cost IoT Weather Station for Smart Agriculture*. E3S Web of Conferences. https://www.e3s-conferences.org/articles/e3sconf/pdf/2023/28/e3sconf_icmed-icmpc2023_01142.pdf
- Sharma, P., & Gupta, R. (2023). *A Review on IoT-based Weather Monitoring Systems for Smart Cities*. IEEE Xplore. <https://ieeexplore.ieee.org/document/9726106/>